/cat/ is an example of a simple pattern. It matches any string that contains the substring cat. In fact, inside a regular expression pattern, all characters except ., |, (,), [,], {, }, +, \, ^, $, *, and ? match themselves. So, at the risk of creating something that sounds like a logic puzzle, here are some patterns and examples of strings they match and don't match:

/cat/	Matches "dog and cat" and "catch" but not "Cat" or "c.a.t."
/123/	Matches "86512312" and "abc123" but not "1.23"
/t a b/	Matches "hit a ball" but not "table"

If you want to match one of the special characters literally in a pattern, precede it with a backslash, so /*/ is a pattern that matches a single asterisk, and /\/ is a pattern that matches a forward slash. Those backslashes can be confusing, so Ruby provides a %r delimiter for regular expressions, similar to %q for strings. The recommended delimiter is the curly brace, so you can write regular expression literals as %r{cat} or %r{\/}.

Regular expression literals are processed like double-quoted strings. In particular, you can use #{...} expression interpolations in the pattern.

Matching Strings with Patterns

The Ruby operator =~ matches a string against a pattern. It returns the character offset of the string at which the beginning of the match occurred:

```
/cat/ =~ "dog and cat" # => 8
/cat/ =~ "catch"       # => 0
/cat/ =~ "Cat"         # => nil
```

If you only want the boolean true or false of if the match occurred, and don't need the character offset, you can use the match? method. The use of match? is more common than the operator in current Ruby style.

```
/cat/.match?("dog and cat") # => true
/cat/.match?("catch")       # => true
/cat/.match?("Cat")         # => false
```

You can put the string on the left hand side of either the =~ operator or the .match? method if you prefer.

```
"dog and cat" =~ /cat/ # => 8
"catch" =~ /cat/       # => 0
"catch".match?(/cat/)  # => true
"Cat" =~ /cat/         # => nil
```

Because pattern matching returns nil when it fails and because nil is equivalent to false in a boolean context, you can use the result of a pattern match as a condition in statements such as if and while.

```
str = "cat and dog"

if str.match?(/cat/)
  puts "There's a cat here somewhere"
end
```

produces:

```
There's a cat here somewhere
```